

NumPy-Based CNN Implementation for MNIST Handwritten Digit Classification

Advaith Subramanian Sahasranamam

Intro to Deep Learning

Dr. Bo Yuan

Final Project — Type B

Abstract—This project presents the design and implementation of a Convolutional Neural Network (CNN), written in Python from scratch using NumPy for the MNIST handwritten digit classification. This network consists of a single convolutional layer, ReLU activation function, max pooling, and a fully connected dense layer which was trained with a softmax cross-entropy loss. Motivated by the goal of developing an understanding of the inner workings of deep learning systems, including forward and backward propagation, this project did not rely on any deep learning frameworks. This model was trained for 20 epochs, and used a batch size of 32. With stable convergence demonstrated, the average training loss decreased from 0.755 to 0.084, and the model achieved a train accuracy of 97.52% and a test accuracy of 95.95%. This project demonstrated that fundamental CNN components can be manually implemented, while producing competitive performance.

I. INTRODUCTION

Deep learning has become one of the most powerful tools in the history of machine learning, dominating its modern landscape. With Convolutional Neural Networks (CNNs) being very effective at image classification, frameworks such as TensorFlow and PyTorch have made it easier to build and deploy such models. These frameworks, however, often abstract the underlying mathematics. While these tools have shown remarkable success in applications, their “black-box” capability often obscures their inner workings.

The goal of this project was to build a CNN from scratch, using only the Python and NumPy libraries — relying on no automatic gradient calculation. All components, including the forward passes, backpropagation and loss computation, were implemented by-hand. By doing so, the project aimed to develop a concrete understanding of how CNNs work from a mathematical and calculus perspective.

The MNIST dataset was chosen as the benchmark for this project, as it is a well-established dataset for handwritten digit classification. It consists of 70,000 grayscale images across 10 classes. The model was trained and evaluated on this dataset to validate that a manually implemented CNN could still achieve competitive performance.

II. RELATED WORK

The MNIST dataset, introduced by LeCun et al. (1988) [1], has served as a foundational and reliable benchmark in the development of CNNs. In their seminal work, LeCun et al. proposed the LeNet-5, one of the earliest CNN architectures, which demonstrated that locally connected layers with shared

weights could significantly outperform fully connected neural networks on image classification tasks. LeNet-5 achieved a test error rate of less than 1% on MNIST, which set a strong baseline for future work.

Since then, a wide variety of architectures were proposed and deployed, such as the AlexNet, introduced by Krizhevsky et al. [2]. It demonstrated the effectiveness of deep CNNs trained on GPUs for large-scale image recognition. While AlexNet was designed for the more complex ImageNet dataset, its core principles of stacked convolutional layers, ReLU activation functions, and max pooling directly influenced how simpler models like the one in this project are structured. More recent work showed that simple CNNs with basic data augmentation and careful training could achieve near state-of-the-art results on MNIST without complex architectures [3]. This supports the motivation for this project, which prioritizes simplicity over architectural complexity, incorporating only the basic elements of CNNs — and well-chosen training parameters.

III. DATA DESCRIPTION

The dataset used in this project is the MNIST dataset [11], which is a widely used benchmark for handwritten digit classification. It consists of 70,000 grayscale images each of size 28×28 pixels, representing 10 classes of digits 0 through 9. This dataset is also split into 60,000 training images and 10,000 test images.

Each pixel value is represented as an integer value of the range $[0, 255]$, where 0 corresponds to black, and 255 to white. Prior to the training, the pixel values were normalized to the range $[0, 1]$ by dividing by 255, which helps stabilize gradient updates during training. The images are of single channel, representing only grayscale. To be compatible with the convolutional layer, each image was reshaped to have an explicit channel dimension, resulting in an input shape of $(N, 1, 28, 28)$ — with N representing the batch size. The class labels are represented as integers in the range $[0, 9]$ — corresponding to their ground truth.

In this project, no data augmentation was used. However, the data was shuffled prior to training, increasing the model's chances of receiving uniform exposure to different samples.

IV. METHOD DESCRIPTION

This project implements a CNN using a sequence of differentiable operations, each with a defined forward and backward pass to support end-to-end training via backpropagation. It was also ensured that the dimensions of vectors and matrices were compatible within the step, and for all other layers to work with.

A. Convolution

The convolutional layer applies F learnable filters (kernels) of size $K \times K$ across the input. For an input $\mathbf{X}$ of shape (N, C, H, W) and filter $\mathbf{W}_f$, the output feature map is computed as:

$$Y(n, f, i, j) = \sum_c \sum_m \sum_k X(n, c, i+m, j+k) \cdot W(f, c, m, k) + b_f \quad (1)$$

where m, k index over the kernel dimensions. With $K=3$, no padding, and stride 1, an input of 28×28 produces output feature maps of size 26×26 . During backpropagation, the gradients with respect to the weights, biases, and input are:

$$\frac{\partial L}{\partial W(f, c, m, k)} = \sum_n \sum_i \sum_j X(n, c, i+m, j+k) \cdot \frac{\partial L}{\partial Y(n, f, i, j)} \quad (2)$$

$$\frac{\partial L}{\partial b_f} = \sum_n \sum_i \sum_j \frac{\partial L}{\partial Y(n, f, i, j)} \quad (3)$$

$$\frac{\partial L}{\partial X(n, c, i+m, j+k)} = \sum_f W(f, c, m, k) \cdot \frac{\partial L}{\partial Y(n, f, i, j)} \quad (4)$$

Weights were updated via gradient descent, using the formula $\mathbf{W} \leftarrow \mathbf{W} - \alpha \cdot \frac{\partial L}{\partial \mathbf{W}}$, where α is the learning rate [4], [8].

B. ReLU Activation

The ReLU activation function is applied element-wise after the convolutional layer:

$$f(x) = \max(0, x) \quad (5)$$

During backpropagation, the gradient is gated by the activation mask:

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} \cdot \mathbb{1}[x > 0] \quad (6)$$

Here, $\mathbb{1}[x > 0]$ is an indicator that is 1 where the pre-activation input was positive — and set to 0 elsewhere [5].

C. Max Pooling

A 2×2 max pooling operation reduces the spatial dimensions of the feature maps by retaining the maximum value within each non-overlapping pooling window:

$$Y(n, c, i, j) = \max_{(p,q) \in P} X(n, c, i \cdot P + p, j \cdot P + q) \quad (7)$$

where $P=2$ is the pool size. This reduces the 26×26 feature maps to 13×13 . During backpropagation, gradients are routed

only to the position of the maximum value in each window, and were set to zero elsewhere. The general rule is as follows:

$$\frac{\partial L}{\partial X(n, c, i \cdot P + p, j \cdot P + q)} = \frac{\partial L}{\partial Y(n, c, i, j)} \cdot \mathbb{1}[(p, q) = \arg\max \text{ of patch}] \quad (8)$$

D. Flatten

The output of the pooling layer, of shape $(N, 8, 13, 13)$, is reshaped into a 1D vector of size $8 \times 13 \times 13 = 1,352$ per sample. The backward pass simply reshapes the incoming gradient back to $(N, 8, 13, 13)$.

E. Fully Connected Layer

The flattened vector is passed through a fully connected layer with 10 output neurons, one per class. This is modelled as $\mathbf{z} = \mathbf{x}\mathbf{W} + \mathbf{b}$, where $\mathbf{x} \in \mathbb{R}^{1352}$, $\mathbf{W} \in \mathbb{R}^{1352 \times 10}$, and $\mathbf{b} \in \mathbb{R}^{10}$. The gradients during backpropagation are:

$$\frac{\partial L}{\partial \mathbf{W}} = \mathbf{x}^\top \cdot \frac{\partial L}{\partial \mathbf{z}}, \quad \frac{\partial L}{\partial \mathbf{b}} = \sum_n \frac{\partial L}{\partial \mathbf{z}}, \quad \frac{\partial L}{\partial \mathbf{x}} = \frac{\partial L}{\partial \mathbf{z}} \cdot \mathbf{W}^\top \quad (9)$$

The output logits are passed through a numerically stable softmax to produce class probabilities [7]:

$$p_k = \frac{\exp(z_k - \max(\mathbf{z}))}{\sum_j \exp(z_j - \max(\mathbf{z}))} \quad (10)$$

The cross-entropy loss is then computed over the batch:

$$L = -\frac{1}{N} \sum_n \log(p(y_n) + \varepsilon) \quad (11)$$

where y_n is the true class label and $\varepsilon = 10^{-12}$ is added for numerical stability. The gradient of the combined softmax cross-entropy with respect to the logits simplifies to:

$$\frac{\partial L}{\partial z_k} = \frac{1}{N} (p_k - \mathbb{1}[k = y_n]) \quad (12)$$

This gradient is then used to conduct full backpropagation through the network [6].

V. MODEL DESCRIPTION

The model implemented for this project is of a lightweight CNN architecture, designed specifically for the 10-class digit classification. The architecture is kept intentionally simple, with one convolutional layer followed by a fully-connected output layer. This was intentionally done since NumPy does not directly offer GPU support — and hence, a simpler model would also provide a faster training time.

The model's architecture has been summarized in Table I.

The convolutional layer has 8 filters of size 3×3 , giving a total of $(8 \times 1 \times 3 \times 3) + 8 = 80$ trainable parameters. The fully connected layer contains $(1352 \times 10) + 10 = 13,530$ trainable parameters. The total number of trainable parameters in the network is 13,610.

TABLE I
MODEL ARCHITECTURE SUMMARY

Layer	Configuration	Output Shape
Input	Grayscale image	$(N, 1, 28, 28)$
Convolutional	8 filters, 3×3 , stride 1, no pad	$(N, 8, 26, 26)$
ReLU	Element-wise activation	$(N, 8, 26, 26)$
Max Pooling	2×2 pool size	$(N, 8, 13, 13)$
Flatten	—	$(N, 1352)$
Fully Connected	$1352 \rightarrow 10$ units	$(N, 10)$
Softmax CE Loss	10-class output	—

At the beginning, all weights were initialized using a random normal initialization, which was scaled by 0.1 — and all biases were initialized to zero. The model was trained using mini-batch gradient descent with a learning rate of $\alpha = 0.02$ and with a batch size of 32 over 20 epochs on 30,000 training samples.

VI. EXPERIMENTAL PROCEDURE AND RESULTS

A. Setup

The model was implemented in Python using NumPy, and trained in a Google Colab environment. The MNIST dataset was loaded from the TensorFlow’s Keras API. Here, TensorFlow was only used for data access, with none of the model implementation using any of its components — all components were implemented manually. The training set was sub-sampled with 30,000 training images. Post training, an evaluation of the training accuracy was performed on 5,000 training samples and the test accuracy was evaluated over 2,000 test samples.

B. Training Loss

The average cross-entropy loss per epoch is reported in Table II. There was a consistent and monotonic decrease in the loss per epoch, dropping significantly in the early epochs and decreasing steadily thereafter. This indicates that the convergence was stable, with no signs of divergence or instability.

C. Results

After 20 epochs of training, the model was evaluated on both the training and test sets. The achieved evaluation scores are summarized in Table III.

A training accuracy of 97.52% was reached, and a test accuracy of 95.95% was also achieved — which surpasses the 93% minimum threshold. This suggests that the model generalized well despite its simplicity, with no significant overfitting being obvious. This result is competitive with other lightweight CNN baselines reported on MNIST, and demonstrated that a manually implemented CNN with 13,610 trainable parameters is sufficient to achieve strong performance on the MNIST benchmark. Furthermore, all LLM generated outputs were independently verified by the author before being incorporated into the work.

TABLE II
AVERAGE CROSS-ENTROPY LOSS PER EPOCH

Epoch	Loss
1	0.7554
2	0.3457
3	0.3051
4	0.2711
5	0.2412
6	0.2115
7	0.1886
8	0.1681
9	0.1530
10	0.1396
11	0.1294
12	0.1211
13	0.1132
14	0.1074
15	0.1022
16	0.0978
17	0.0937
18	0.0902
19	0.0870
20	0.0837

TABLE III
CLASSIFICATION ACCURACY

Split	Accuracy
Train (5,000 samples)	97.52%
Test (2,000 samples)	95.95%

VII. CONCLUSION

This project demonstrated that a Convolutional Neural Network can be built from scratch using only Python and NumPy, without relying on any deep learning frameworks. Despite the lack of reliance of any frameworks, the results also demonstrated that the performance on the MNIST benchmark was competitive. By implementing every component manually, including the convolutional layer, ReLU, max pooling, flatten, fully connected layer, softmax cross-entropy loss, and back-propagation — this project developed a concrete, low-level understanding of the functioning of CNNs — solidifying the mathematical understanding.

The model, which only consisted of 13,610 trainable parameters, achieved a test accuracy of 95.95% after 20 epochs of training, with consistent reduction in loss throughout the training phase. The small generalization gap between training and test accuracy suggests that the model was not significantly overfit, which is a notable observation, given that no regularization techniques such as dropout or weight decay were used.

There are, however, directions in which this work can be extended. Adding additional convolutional layers, increasing the number of filters, and experimenting with different momentum-based optimizers such as Adam, can potentially increase performance significantly, producing almost state-of-the-art results. Furthermore, training on the full 60,000 sample training set was not feasible without GPU support, and is noted as a limitation in this study.

Overall, this project confirmed that the fundamental components of Deep Learning are both tractable and practically implementable from scratch, and that even a minimal architecture, when optimized, is capable of producing good results on well-established benchmarks.

NOTES ON REPRODUCIBILITY

Since this project uses random shuffling and initialization of random weights, a seed was not set before the conduction of the experiments. Hence, some of the parameters and evaluation metrics may differ when being reproduced.

AI TOOLS DISCLOSURE

This project made use of AI assistance in two capacities. ChatGPT was used during the implementation phase to develop an understanding of how backpropagation and layer-wise gradients must be derived in a CNN and why, as well as to clarify the motivation for combining the softmax and cross-entropy loss into a single operation. ChatGPT was also used to conduct debugging and fix any errors that occurred. Claude (Anthropic) was used during the writing phase to assist with structuring, drafting, and refining this report, including identifying key points to include in each section, and rendering the rough draft into an IEEE format on Overleaf. All code was written and understood by the author, and all results were produced independently. Furthermore, all LLM outputs were independently verified by the author before being incorporated into the results.

REFERENCES

- [1] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [2] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," *Advances in Neural Information Processing Systems*, vol. 25, pp. 1097–1105, 2012.
- [3] P. Y. Simard, D. Steinkraus, and J. C. Platt, "Best practices for convolutional neural networks applied to visual document analysis," in *Proc. 7th Int. Conf. Document Analysis and Recognition*, 2003, pp. 958–963.
- [4] A. Karpathy, "CS231n Convolutional Neural Networks for Visual Recognition," Stanford University. [Online]. Available: <https://cs231n.github.io/convolutional-networks/>. [Accessed: May 13, 2026].
- [5] A. Karpathy, "CS231n: Backpropagation, Intuitions," Stanford University. [Online]. Available: <https://cs231n.github.io/optimization-2/>. [Accessed: May 13, 2026].
- [6] A. Karpathy, "CS231n Neural Networks Part 2: Setting up the Data and the Loss," Stanford University. [Online]. Available: <https://cs231n.github.io/neural-networks-2/>. [Accessed: May 13, 2026].
- [7] E. Bendersky, "The Softmax function and its derivative," 2016. [Online]. Available: <https://eli.thegreenplace.net/2016/the-softmax-function-and-its-derivative/>. [Accessed: May 13, 2026].
- [8] T. Parr and J. Howard, "The Matrix Calculus You Need For Deep Learning," *explained.ai*. [Online]. Available: <https://explained.ai/matrix-calculus/index.html>. [Accessed: May 13, 2026].
- [9] V. Zhou, "CNNs, Part 1: An Introduction to Convolutional Neural Networks," 2019. [Online]. Available: <https://victorzhou.com/blog/intro-to-cnns-part-1/>. [Accessed: May 13, 2026].
- [10] H. S. A. et al., "Neural Networks from Scratch," *nnfs.io*. [Online]. Available: <https://nnfs.io>. [Accessed: May 13, 2026].
- [11] Keras Team, "MNIST digits classification dataset," *Keras Documentation*. [Online]. Available: <https://keras.io/api/datasets/mnist/>. [Accessed: May 13, 2026].